NET-NEUTRAL AI

Backend Schema & ER Diagram

Database design, table specifications, constraints, and indexes

GitHub DevDays Hackathon 2026 | Version 1.0 | May 2026

Companion Docs	PRD v1.0 + TRD v1.0 + App Flow v1.0
Database	SQLite 3 — single file (database.db)
Tables	2 — credits, rounds
Owner	IT team member
Schema Version	v1.0 — stable, no migrations expected for prototype

1. Overview

The Net-Neutral AI backend uses a single SQLite database file (database.db) stored on the coordinator machine. It persists two categories of data: contribution records per client per round (credits table), and round-level performance history (rounds table). No other data is persisted to disk — registered clients are hardcoded, weight files are deleted after FedAvg, and the global model is stored as a flat .pt file, not in the database.

> **Why SQLite and not PostgreSQL or MongoDB**
>
> SQLite requires zero installation, zero configuration, and zero running services. It is a single file. For a demo running on a local WiFi network with 3 clients and 5 rounds, it is architecturally correct — not a compromise. Using PostgreSQL here would be complexity theatre.

Property	Decision
Database engine	SQLite 3 (built into Python standard library — no pip install needed)
File location	coordinator/database.db (auto-created on first server start)
Access pattern	Single writer (coordinator server only) — no concurrent write conflicts
Backup strategy	Copy database.db after each demo session for future analysis
ORM	None — raw SQL via Python sqlite3 module. Keeps it transparent and debuggable.
Migration plan	None for prototype — schema is stable. Drop and recreate if needed.

2. Entity Relationship Diagram

The diagram below shows both tables, their columns with data types, primary keys, and the relationship between them.

Reading the diagram

The Coordinator writes to both tables after each round. The credits table stores one row per client per round (N rows per round). The rounds table stores one row per round (1 row per round). The 'round' field in credits matches the 'round_number' field in rounds — this is the logical link between the two tables, enforced in application code rather than as a foreign key constraint (SQLite supports this but we keep it simple for the prototype).

3. Table Specifications

3.1 credits

One row inserted per client per round. This is the primary record of compute contribution and reward.

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-incrementing unique row identifier
client_id	TEXT	NOT NULL	Hardcoded client name — 'client_A', 'client_B', 'client_C'
round	INTEGER	NOT NULL	Round number this record belongs to — matches rounds.round_number
samples_trained	INTEGER	NOT NULL DEFAULT 0	Number of training samples processed by this client this round

time_seconds	REAL	NOT NULL DEFAULT 0.0	Wall-clock seconds the client spent on local training
points_earned	INTEGER	NOT NULL DEFAULT 0	Computed as floor(samples_trained / 5) — inserted by coordinator
timestamp	DATETIME	DEFAULT CURRENT_TIMESTAMP	Server time when coordinator received this client's submission

CREATE TABLE statement — credits

```
CREATE TABLE IF NOT EXISTS credits (
    id              INTEGER  PRIMARY KEY AUTOINCREMENT,
    client_id       TEXT     NOT NULL,
    round           INTEGER  NOT NULL,
    samples_trained INTEGER  NOT NULL DEFAULT 0,
    time_seconds    REAL     NOT NULL DEFAULT 0.0,
    points_earned   INTEGER  NOT NULL DEFAULT 0,
    timestamp       DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Indexes — credits

```
-- Fast leaderboard queries (sum credits per client)
CREATE INDEX IF NOT EXISTS idx_credits_client ON credits (client_id);

-- Fast round-specific queries (who submitted this round)
CREATE INDEX IF NOT EXISTS idx_credits_round ON credits (round);
```

Sample data — credits (after 2 rounds, 3 clients)

id	client_id	round	samples_trained	time_seconds	points_earned	timestamp
1	client_A	1	5000	28.3	1000	2026-06-01 10:03:14
2	client_B	1	5000	31.7	1000	2026-06-01 10:03:21
3	client_C	1	5000	29.1	1000	2026-06-01 10:03:18
4	client_A	2	5000	27.9	1000	2026-06-01 10:07:44
5	client_B	2	5000	30.2	1000	2026-06-01 10:07:51
6	client_C	2	5000	28.8	1000	2026-06-01 10:07:48

3.2 rounds

One row inserted per round, after FedAvg and evaluation complete. This is the performance history — useful for plotting accuracy curves after the hackathon.

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-incrementing unique row identifier
round_number	INTEGER	NOT NULL UNIQUE	Round index — 1 through 5 for the prototype
started_at	DATETIME	NOT NULL	Server time when Operator typed start_round
completed_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Server time when FedAvg + evaluation finished
clients_submitted	INTEGER	NOT NULL DEFAULT 0	How many clients submitted weights this round (max 3)
global_accuracy	REAL	NOT NULL DEFAULT 0.0	Global model accuracy on validation set after this round (0.0–1.0)
accuracy_delta	REAL	DEFAULT 0.0	Accuracy change vs previous round — negative means regression

CREATE TABLE statement — rounds

```
CREATE TABLE IF NOT EXISTS rounds (
    id                INTEGER  PRIMARY KEY AUTOINCREMENT,
    round_number      INTEGER  NOT NULL UNIQUE,
    started_at        DATETIME NOT NULL,
    completed_at      DATETIME DEFAULT CURRENT_TIMESTAMP,
    clients_submitted INTEGER  NOT NULL DEFAULT 0,
    global_accuracy   REAL     NOT NULL DEFAULT 0.0,
    accuracy_delta    REAL     DEFAULT 0.0
);
```

Sample data — rounds (after 5 complete rounds)

id	round_number	started_at	completed_at	clients_submitted	global_accuracy	accuracy_delta
1	1	2026-06-01 10:00:00	2026-06-01 10:03:45	3	0.714	+0.041
2	2	2026-06-01 10:05:00	2026-06-01 10:08:12	3	0.738	+0.024

3	3	2026-06-01 10:10:00	2026-06-01 10:13:28	3	0.759	+0.021
4	4	2026-06-01 10:15:00	2026-06-01 10:18:05	3	0.774	+0.015
5	5	2026-06-01 10:20:00	2026-06-01 10:23:31	3	0.791	+0.017

4. Key SQL Queries

These are the exact queries the coordinator runs during operation. IT should implement these in credits.py.

4.1 Insert credit record after client submits

```
INSERT INTO credits
    (client_id, round, samples_trained, time_seconds, points_earned)
VALUES
    (?, ?, ?, ?, ?);

-- Parameters: client_id, round_number, samples, time_secs, floor(samples/5)
```

4.2 Insert round record after FedAvg completes

```
INSERT INTO rounds
    (round_number, started_at, clients_submitted, global_accuracy, accuracy_delta)
VALUES
    (?, ?, ?, ?, ?);

-- accuracy_delta = global_accuracy - previous round's global_accuracy
-- For round 1: accuracy_delta = global_accuracy - pretrained_baseline_accuracy
```

4.3 Leaderboard — cumulative credits per client

```
SELECT
    client_id,
    SUM(points_earned)    AS total_points,
    SUM(samples_trained)  AS total_samples,
    COUNT(*)              AS rounds_participated
FROM credits
GROUP BY client_id
ORDER BY total_points DESC;
```

4.4 Accuracy history — for final summary printout

```
SELECT
    round_number,
    ROUND(global_accuracy * 100, 1)  AS accuracy_pct,
    ROUND(accuracy_delta   * 100, 1) AS delta_pct,
    clients_submitted
FROM rounds
ORDER BY round_number ASC;
```

4.5 Check if all clients submitted this round

```
SELECT COUNT(DISTINCT client_id) AS submitted_count
FROM credits
WHERE round = ?;

-- If submitted_count >= 3: proceed with FedAvg
-- If submitted_count < 3 and timeout expired: proceed with partial FedAvg
```

5. Database Initialisation

The coordinator runs this once on startup — before any client connects. If database.db already exists (e.g. from a previous dry run), it skips creation safely due to IF NOT EXISTS.

```
# coordinator/credits.py

import sqlite3
import os

DB_PATH = os.path.join(os.path.dirname(__file__), 'database.db')

def init_db():
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    cursor.executescript('''
        CREATE TABLE IF NOT EXISTS credits (
            id              INTEGER  PRIMARY KEY AUTOINCREMENT,
            client_id       TEXT     NOT NULL,
            round           INTEGER  NOT NULL,
            samples_trained INTEGER  NOT NULL DEFAULT 0,
            time_seconds    REAL     NOT NULL DEFAULT 0.0,
            points_earned   INTEGER  NOT NULL DEFAULT 0,
            timestamp       DATETIME DEFAULT CURRENT_TIMESTAMP
        );
        CREATE TABLE IF NOT EXISTS rounds (
```

```
            id                 INTEGER  PRIMARY KEY AUTOINCREMENT,
            round_number       INTEGER  NOT NULL UNIQUE,
            started_at         DATETIME NOT NULL,
            completed_at       DATETIME DEFAULT CURRENT_TIMESTAMP,
            clients_submitted  INTEGER  NOT NULL DEFAULT 0,
            global_accuracy    REAL     NOT NULL DEFAULT 0.0,
            accuracy_delta     REAL     DEFAULT 0.0
        );
        CREATE INDEX IF NOT EXISTS idx_credits_client ON credits (client_id);
        CREATE INDEX IF NOT EXISTS idx_credits_round  ON credits (round);
    ''')

    conn.commit()
    conn.close()
    print('[DB] Database initialised at', DB_PATH)
```

6. Post-Hackathon Use of This Data

This is why the rounds table exists. After the demo, you have a complete accuracy history you can use for:

- Plotting a training curve (round vs accuracy) for your GitHub README — shows federated learning working visually
- Analysing how much each client contributed relative to their hardware speed (time_seconds column)
- Identifying whether any rounds had missing clients (clients_submitted < 3) and what the accuracy impact was
- Writing up results for a future research poster or project report

How to export for analysis

After your demo recording, run: sqlite3 database.db ".headers on" ".mode csv" ".output results.csv" "SELECT * FROM rounds;" ".quit" — this gives you a CSV you can drop straight into Excel or matplotlib.

7. Data Integrity Rules

These are enforced in application code in credits.py, not as database constraints. Simpler to debug during build week.

Rule	Where Enforced	What Happens If Violated
points_earned must equal floor(samples/5)	credits.py	Coordinator recomputes — never trusts client-reported points

One credits row per client per round	credits.py	Coordinator checks before insert — duplicate submission rejected
round_number must be sequential	server.py	round_number = previous + 1, always. No gaps allowed.
accuracy_delta = current - previous	credits.py	Computed fresh each round from rounds table — never passed in by client
global_accuracy between 0.0 and 1.0	credits.py	Values outside range logged as error, round still saved